МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике
ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ __**Бушмакина_Кристина_Сергеевна**___
(подпись) (фамилия, имя, отчество)
Группы ____**ИПо-41**____

Руководитель практической подготовки от профильной организации

_____**Генеральный_директор**_____
(должность)

_____ _____**Середняков_С.Д.**_____ (подпись) (фамилия, имя, отчество)

МП «13» ноября 2025 года
Руководитель практической подготовки от института

_____ _____
(подпись) (фамилия, имя, отчество)

«13» ноября 2025 года

Пермь, 2025 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3
РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 7
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ 7
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 10
ЗАКЛЮЧЕНИЕ 43
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 46

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

Разработка программного обеспечения играет ключевую роль в современном информационном мире, обеспечивая автоматизацию бизнес-процессов, эффективное управление данными и повышение доступности сервисов. Современные приложения позволяют компаниям оперативно реагировать на изменения рынка, улучшать взаимодействие с клиентами и принимать решения на основе актуальной информации.

В рамках данной работы была разработана система для сервиса доставки готовых обедов — информационная система, центром которой является база данных, реализованная в PostgreSQL. Разработка ПО позволила создать структурированную, безопасную и масштабируемую платформу для управления товарами (готовыми блюдами), заказами и пользователями с различными ролями. Реализация нормализованной схемы, триггеров, функций и механизмов контроля доступа обеспечила целостность данных и поддержку бизнес-логики, такой как запрет удаления товара, присутствующего в заказах, или динамическое отображение остатков.

Таким образом, разработка данного программного решения демонстрирует, как правильно построенное ПО становится основой для прозрачного, надёжного и удобного управления деятельностью компании в условиях высоких требований к скорости и точности обработки информации.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения.

Цель практики:

Закрепление и углубление теоретических знаний, полученных в техникуме в области баз данных и разработки программного обеспечения, а также приобретение практических навыков проектирования, создания и администрирования реляционных баз данных в СУБД PostgreSQL, разработки логики приложения на основе ASP.NET Core Web API, реализации ролевой модели доступа, обеспечения целостности данных и построения корректной архитектуры информационной системы на примере создания сервиса доставки готовых обедов.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Изучить IT-инфраструктуру предприятия и роль программного обеспечения в его бизнес-процессах.

2. Ознакомиться с используемым программным обеспечением (СУБД, средствами разработки и администрирования).

3. Принять участие в реальных задачах, связанных с программным обеспечением

4. Приобрести навыки разработки требований к программным модулям на основе анализа проектной и технической документации;

5. Выполнять моделирование процесса разработки программного обеспечения;

6. Освоить принципы обеспечения безопасности, резервного копирования и восстановления данных;

7. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

8. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности;

9. Планировать и реализовывать собственное профессиональное и личностное развитие;

10. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами;

11. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста;

12. Использовать информационные технологии в профессиональной деятельности;

13. Пользоваться профессиональной документацией на государственном и иностранном языках

Место прохождения практики: ООО Эльбрус

Сроки прохождения практики: с 03 по 13 ноября 2025 г.

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В рамках производственной практики в ООО «Эльбрус» я работала программистом в офисе, занимаясь поддержкой и анализом баз данных (PostgreSQL), используемых для учёта заказов, производства и логистики. Мои обязанности включали участие в разработке и тестировании хранимых процедур, мониторинг производительности БД, обеспечение целостности данных и соблюдение стандартов информационной безопасности — включая резервное копирование и защиту от несанкционированного доступа.

Техника безопасности на моём рабочем месте ограничивалась требованиями к работе с электронной техникой: запрет на использование повреждённых кабелей или розеток, недопущение перегрузки электросетей, запрет на работу с мокрыми руками, а также обязательное отключение оборудования при неисправностях. Самостоятельный ремонт техники запрещён — все обращения направлялись в службу ИТ-поддержки. Также соблюдался режим проветривания и порядок хранения документов, включая защиту конфиденциальной информации.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

В ходе прохождения производственной практики в ООО «Эльбрус» я работала программистом в небольшом, но функционально налаженном офисе. В компании всего три постоянных сотрудника: генеральный директор, который также курирует производство, и два специалиста — отвечаеют за логистику и бухгалтерию. Второму я подчинялась в течение практики.

Режим работы — пятидневный, с 9:00 до 18:00, с часовым перерывом. Правила внутреннего распорядка были простыми, но чёткими: приход вовремя, бережное отношение к технике, запрет на использование рабочих устройств в личных целях и обязательная конфиденциальность данных. Так как компания небольшая, все сотрудники знают друг друга и взаимодействуют напрямую — это позволило мне быстро включиться в процессы.

Мои обязанности включали поддержку базы данных PostgreSQL: написание и доработку хранимых процедур, проверку корректности данных между отделами продаж, производства и склада, а также участие в резервном копировании и мониторинге производительности. Задачи ставились устно или через простые чаты — без сложных систем учёта, но с чётким контролем: каждый код проверялся, каждая правка документировалась. Я участвовала в обсуждениях с другими сотрудниками, чтобы понять, как данные влияют на реальные производственные процессы — это помогло мне увидеть связь между программированием и бизнесом.

Практика в такой мини-компании дала мне ценный опыт: я научилась работать автономно, быстро адаптироваться, отвечать за результат и

понимать, как даже в маленькой организации важна точность и дисциплина в работе с данными.

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

Права и обязанности сотрудника. Должностная инструкция программиста

В ходе прохождения производственной практики в ООО «Эльбрус» моя деятельность как практиканта, выполняющего функции программиста, осуществлялась в рамках устной договорённости и внутреннего регламента компании, поскольку на предприятии отсутствует официально утверждённая должностная инструкция в бумажном виде. Однако круг моих обязанностей был чётко определён руководителем — специалистом по информационным технологиям, совмещающим функции системного администратора и разработчика.

К основным обязанностям относились:

- Поддержка и развитие корпоративной базы данных на платформе PostgreSQL;
- Разработка, тестирование и внедрение хранимых процедур и триггеров для автоматизации бизнес-процессов (учёт заказов, движение материалов, статусы производства);
- Обеспечение целостности и согласованности данных между отделами;
- Участие в резервном копировании баз данных и проверке работоспособности восстановления;
- Документирование изменений в структуре БД и коде;

- Соблюдение правил информационной безопасности при доступе к данным и работе с серверами.

Согласно установленному регламенту, я имела право:

- Получать необходимую информацию и доступ к базам данных для выполнения поставленных задач (в рамках предоставленных прав доступа);
- Задавать вопросы руководителю и запрашивать разъяснения по техническим требованиям;
- Использовать служебное оборудование и программное обеспечение в целях выполнения трудовых обязанностей;
- Вносить предложения по оптимизации процессов обработки и хранения данных.

При этом моя деятельность регулировалась принципами добросовестного использования ресурсов предприятия, конфиденциальности информации и ответственного отношения к результату работы. Нарушение регламентов, включая попытки несанкционированного доступа или модификации данных без согласования, строго запрещено.

Хотя формальной должности «программист» в штатном расписании нет, фактически выполняемые мной функции соответствуют этой роли в контексте малого предприятия. Деятельность координировалась напрямую с ответственным IT-специалистом, который также контролировал качество и сроки выполнения задач.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Ознакомление: с организацией информационного обеспечения подразделения;

Производственная характеристика и показатели деятельности предприятия

Практика проходила на базе Эльбруса — динамично развивающейся компании, специализирующейся на производстве и реализации высококачественных офисных перегородок и интерьерных решений. Предприятие оснащено современным оборудованием для раскроя, обработки стекла, алюминия и МДФ, что позволяет выпускать продукцию по индивидуальным проектам. Основными направлениями деятельности являются проектирование, изготовление и монтаж перегородок, а также комплексное оснащение офисных пространств.

В ходе практики были изучены процессы управления заказами, учёта материалов, взаимодействия с поставщиками и логистики. Особое внимание уделено автоматизации бизнес-процессов: на предприятии активно используется программное обеспечение для учёта производства, складских остатков и работы с клиентами, что позволило глубже понять принципы построения информационных систем в условиях реального производства.

Также в рамках анализа существующих ИТ-решений была рассмотрена возможность масштабирования цифровой инфраструктуры компании для поддержки новых направлений бизнеса, включая развитие внутреннего сервиса питания сотрудников и возможный выход на рынок B2C-услуг. На этой основе была выбрана тема разработки информационной системы для **сервиса**

доставки готовых обедов, как примера применения полученных знаний в смежной, но логически схожей предметной области.

Назначение и функции ПО

В рамках практики было проведено проектирование и разработка прототипа программного обеспечения — **базы данных для сервиса доставки готовых обедов**. Целью создания ПО является автоматизация процессов учёта товаров (готовых блюд), управления заказами, контроля остатков и обеспечения многопользовательского доступа с различными уровнями прав.

Система предназначена для внутреннего использования (например, при масштабировании корпоративного питания) или как самостоятельный цифровой продукт для вывода на рынок. Программное обеспечение позволяет:

- Вести каталог блюд;
- Обеспечивать авторизацию пользователей с ролями: гость, клиент, менеджер, администратор;
- Формировать и отслеживать заказы;
- Реализовывать фильтрацию, поиск и сортировку товаров;
- Контролировать доступ к операциям добавления, редактирования и удаления товаров.

Анализ предметной области

Предметная область — **сервис доставки готовых обедов**, где основными сущностями и процессами являются:

- **Пользователи** — лица, взаимодействующие с системой: гости, клиенты, менеджеры, администраторы.

- **Категории блюд** — группировка товаров (основные блюда, салаты, супы и т.д.).
- **Готовые обеды (продукты)** — товары с указанием названия, описания, цены, скидки, количества на складе, производителя и поставщика.
- **Заказы** — оформленные клиентами заявки на доставку одного или нескольких блюд.
- **Состав заказа** — детализация, какие именно блюда и в каком количестве входят в заказ.
- **Производители и поставщики** — организации, обеспечивающие выпуск и поставку продукции.

Ключевые процессы:

- Авторизация пользователя и ограничение функционала по ролям.
- Просмотр и фильтрация ассортимента.
- Учёт остатков на складе.
- Запрет удаления товара, если он присутствует в заказах.
- Хранение истории заказов.

Логическая модель данных

Логическая модель данных представлена в виде ER-диаграммы (см. рисунок ниже). Она построена в соответствии с требованиями третьей нормальной формы и включает следующие основные сущности и связи:

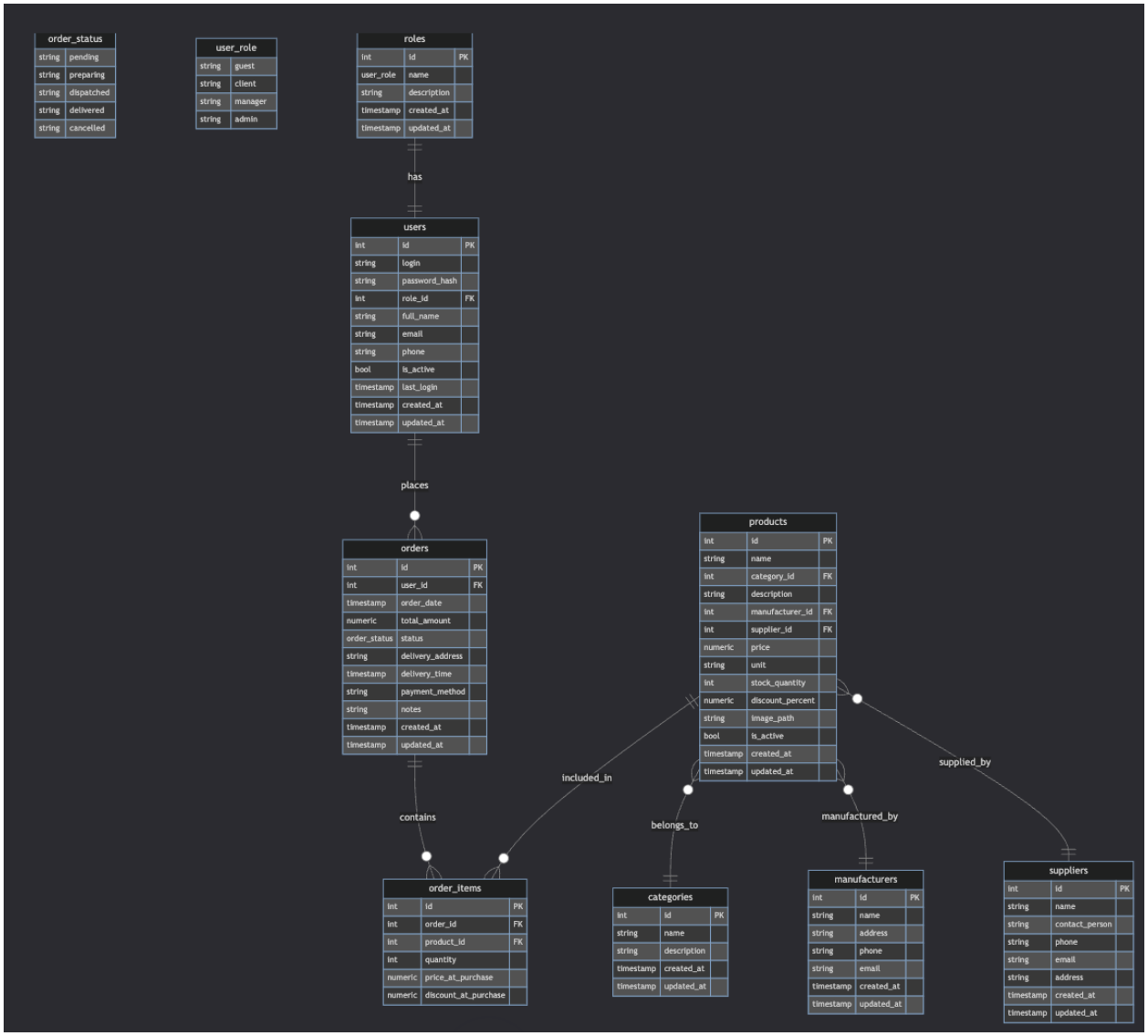
Основные сущности:

- Roles — роли пользователей (гость, клиент, менеджер, администратор).
- Users — пользователи системы; связаны с Roles через внешний ключ.
- Categories — категории блюд.

- Products — готовые обеды; связаны с Categories, Manufacturers, Suppliers.
- Orders — заказы; связаны с Users.
- OrderItems — позиции в заказе; связывают Orders и Products (многие-ко-многим).
- Manufacturers, Suppliers — справочники производителей и поставщиков.

Связи:

- Один пользователь может иметь одну роль, одна роль — у многих пользователей (1:M).
- Одно блюдо относится к одной категории, одна категория содержит множество блюд (1:M).
- Один заказ принадлежит одному пользователю, один пользователь может иметь несколько заказов (1:M).
- Один заказ содержит несколько позиций (OrderItems), каждая из которых ссылается на одно блюдо (M:N через ассоциативную таблицу).
- Удаление товара запрещено, если он есть в OrderItems (ON DELETE RESTRICT).


order_status
string pending
string preparing
string dispatched
string delivered
string cancelled
user_role
string guest
string client
string manager
string admin
roles
int id PK
user_role name
string description
timestamp created_at
timestamp updated_at
has
users
int id PK
string login
string password_hash
int role_id FK
string full_name
string email
string phone
bool is_active
timestamp last_login
timestamp created_at
timestamp updated_at
places
orders
int id PK
int user_id FK
timestamp order_date
numeric total_amount
order_status status
string delivery_address
timestamp delivery_time
string payment_method
string notes
timestamp created_at
timestamp updated_at
products
int id PK
string name
int category_id FK
string description
int manufacturer_id FK
int supplier_id FK
numeric price
string unit
int stock_quantity
numeric discount_percent
string image_path
bool is_active
timestamp created_at
timestamp updated_at
included_in
supplied_by
contains
belongs_to
manufactured_by
order_items
int id PK
int order_id FK
int product_id FK
int quantity
numeric price_at_purchase
numeric discount_at_purchase
categories
int id PK
string name
string description
timestamp created_at
timestamp updated_at
manufacturers
int id PK
string name
string address
string phone
string email
timestamp created_at
timestamp updated_at
suppliers
int id PK
string name
string contact_person
string phone
string email
string address
timestamp created_at
timestamp updated_at

Тема 3.2. Выполнение производственных заданий

3.2.1. Разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент

3.2.1.1. Сбор информации о существующем состоянии продукта:

Постановка задач

В ходе прохождения практики в ООО «Эльбрус» была выявлена потребность в автоматизации внутренних бизнес-процессов, связанных с управлением ресурсами и цифровизацией деятельности компании. Хотя основным направлением предприятия является производство офисных перегородок, передовая ИТ-инфраструктура и интерес к развитию цифровых сервисов позволили рассмотреть возможность расширения бизнеса за счёт смежных направлений — в частности, организации корпоративного питания сотрудников или запуска B2C-сервиса по доставке готовых обедов.

На основе изучения бизнес-процессов и опыта работы с информационными системами было принято решение спроектировать и реализовать прототип **информационной системы для сервиса доставки готовых обедов**. Целью проекта стало:

- Закрепление теоретических знаний по проектированию баз данных;
- Практическое применение навыков разработки ПО на платформе ASP.NET Core Web API;
- Создание нормализованной, безопасной и масштабируемой БД в PostgreSQL;
- Автоматизация ключевых процессов: учёт товаров, управление заказами, авторизация пользователей, контроль остатков.

Описание бизнес-процесса (состояние «AS IS»)

На момент начала проекта система управления ассортиментом и заказами отсутствовала. Предполагалось, что при запуске нового направления (доставка обедов) все процессы будут организованы вручную:

1. **Учёт блюд** — ведётся в Excel-таблице: название, описание, цена, категория, количество порций.
2. **Изображения блюд** — хранятся в папке на общем диске, без привязки к таблице.
3. **Заказы клиентов** — принимаются по телефону или в мессенджерах, фиксируются вручную (в блокноте или чате).
4. **Контроль остатков** — сотрудник ежедневно сверяет продажи с начальными остатками, вносит изменения вручную.
5. **Роли пользователей** — не формализованы; доступ к информации — общий.
6. **Поиск и фильтрация** — невозможны; поиск товара — по памяти или через поиск по файлу.
7. **Безопасность данных** — отсутствует: нет резервного копирования, аудита изменений, защиты от удаления записей.

Проблемы состояния «AS IS»:

- Высокая вероятность ошибок (перепродажа, путаница в заказах);
- Нет актуальной информации об остатках;
- Отсутствие истории заказов;
- Невозможно быстро найти или отфильтровать товар;
- Риск потери данных;
- Нет разделения прав доступа.

Описание процесса модернизации (состояние «TO BE»)

В результате модернизации был разработан прототип **автоматизированной информационной системы** на базе PostgreSQL и ASP.NET Core Web API. Реализовано следующее:

1. **Централизованная база данных**:

a. Все данные о блюдах, заказах, пользователях хранятся в единой БД.

b. Обеспечена ссылочная целостность, нормализация до 3НФ.

2. **Автоматический учёт остатков**:

a. При оформлении заказа количество порций автоматически уменьшается.

b. Если остаток = 0, блюдо помечается как недоступное (визуально — голубым цветом).

3. **Многопользовательский режим с ролями**:

a. Гость — просматривает меню без фильтрации.

b. Клиент — может просматривать товары.

c. Менеджер — видит заказы, фильтрует/сортирует товары.

d. Администратор — добавляет, редактирует, удаляет товары и заказы.

4. **Функции поиска, сортировки и фильтрации**:

a. Менеджер и админ могут искать по всем текстовым полям.

b. Сортировка по остаткам (возрастание/убывание).

c. Фильтрация по поставщику.

d. Все операции — в реальном времени.

5. **Работа с изображениями**:

a. Изображения блюд хранятся в папке приложения.

b. В БД сохраняется путь к файлу.

c. При редактировании старое фото удаляется, новое — загружается с ограничением 300×200 px.

6. **Защита от ошибок**:

a. Нельзя удалить блюдо, если оно есть в заказе (реализовано через ON DELETE RESTRICT и функцию can_delete_product()).

b. Пароли хранятся в хэшированном виде.

c. Триггеры обновляют updated_at при изменении записей.

7. **Интерфейс и логика приложения**:

a. Авторизация через логин/пароль.

b. На экране отображается ФИО пользователя.

c. Поддержка гостевого режима.

d. Визуальная индикация скидок и отсутствия товара.

Как будут решаться задачи, которые необходимо автоматизировать

Задача	**Решение**
Учёт товаров	Реализована таблица products с полями: название, категория, цена, остаток, скидка, изображение.
Контроль остатков	Поле stock_quantity обновляется при создании/редактировании заказа.
Поиск и фильтрация	Через Web API: параметры search, supplierId, sortByStock.
Разделение прав доступа	Роли в ENUM user_role, проверка в контроллерах C#.
Безопасность данных	Хэширование паролей, триггеры, ограничения внешних ключей, резервное копирование (через pg_dump).

Управление заказами	Таблицы orders и order_items, статус-менеджмент через order_status.
Защита от удаления	ON DELETE RESTRICT + проверка через функцию can_delete_product().
Визуальное отображение	Логика подсветки строк (скидка >15% - зелёный, нет на складе → голубой) — на стороне клиента.

Перечень действий процессов «AS IS» и «TO BE» (сравнительный анализ)

Процесс	**AS IS (ручной способ)**	**TO BE (автоматизированный способ)**	**Улучшения**
Учёт блюд	Excel + папка с фото	БД PostgreSQL + Web API	Данные структурированы, связаны, защищены
Просмотр меню	Чтение файла	Онлайн-каталог в браузере	Быстрый доступ, мобильность
Поиск товара	Поиск по файлу или памяти	Поиск по всем полям в реальном времени	Экономия времени, точность
Фильтрация	Невозможна	По поставщику, категории, остаткам	Гибкий анализ ассортимента

Оформление заказа	По телефону / в чате	Через систему с выбором блюд	Исключение ошибок, автоматическое списание
Контроль остатков	Ручной пересчёт	Автоматическое обновление	Актуальные данные, нет перепродаж
Добавление товара	Редактирование Excel	Форма в приложении с загрузкой фото	Удобство, контроль качества изображений
Удаление товара	Удаление строки в Excel	Через интерфейс с проверкой наличия в заказах	Защита от случайного удаления
Безопасность	Низкая	Высокая (роли, хэши, триггеры)	Защита от несанкционированного доступа
Аудит изменений	Отсутствует	Поля created_at, updated_at, триггеры	Возможность отслеживания изменений

3.2.1.2. Разработка технического задания:

Основные требования к техническому заданию

Техническое задание (ТЗ) является ключевым документом в процессе разработки программного обеспечения. Оно определяет цели, функции, состав и условия создания системы.

Основные требования к ТЗ:

- Полнота — все требования к системе должны быть описаны без пропусков.
- Однозначность — формулировки не должны допускать двойного толкования.
- Проверяемость — каждое требование должно быть возможно проверить (например, через тестирование).
- Согласованность — отсутствие противоречий между разделами.
- Модифицируемость — структура должна позволять легко вносить изменения.

Отслеживаемость — каждое требование должно иметь уникальный идентификатор для контроля на всех этапах жизненного цикла ПО.

В рамках практики было разработано ТЗ на прототип информационной системы для сервиса доставки готовых обедов, что позволило закрепить навыки документирования требований и проектирования ПО в условиях, приближенных к реальным.

Стандарты, используемые для написания технического задания (ГОСТ 19 и ГОСТ 34)

Для оформления технической документации на программное обеспечение применяются межгосударственные стандарты серии **ГОСТ 19** и **ГОСТ 34**,

разработанные в рамках Единой системы программной документации (ЕСПД).

ГОСТ 19.201-78

«Техническое задание. Требования к содержанию и оформлению»

— Устанавливает структуру и правила оформления технического задания на программные средства. Включает:

- Наименование разработки,
- Основание для разработки,
- Назначение разработки,
- Требования к программе,
- Стадии и этапы разработки,
- Порядок контроля и приёмки.

ГОСТ 34.601-90

«Автоматизированные системы. Стадии и этапы разработки»

— Определяет жизненный цикл автоматизированной системы:

1. Формирование требований,
2. Разработка концепции,
3. Техническое задание,
4. Эскизный проект,
5. Технический проект,
6. Рабочая документация,
7. Внедрение,
8. Эксплуатация.

Эти стандарты были применены при подготовке ТЗ для проекта, что обеспечило соответствие профессиональным требованиям и упорядоченность документации.

Содержание технического задания

На основе ГОСТ 19.201-78 содержание ТЗ включает следующие основные разделы:

1. **Введение**
    a. Наименование системы: *Информационная система "Сервис доставки готовых обедов"*
    b. Основание для разработки: учебное задание + анализ потребности в автоматизации смежных бизнес-процессов ООО «Эльбрус»
    c. Источники разработки: материалы практики, нормативные документы, примеры существующих решений
2. **Назначение разработки**
    a. Цель: автоматизация учёта товаров, заказов и управления ассортиментом
    b. Область применения: внутреннее корпоративное питание или B2C-сервис доставки
3. **Требования к системе**
    a. Функциональные:
        i. Авторизация пользователей по ролям (гость, клиент, менеджер, администратор)
        ii. Просмотр, добавление, редактирование, удаление товаров
        iii. Учёт остатков на складе

iv. Оформление и просмотр заказов
v. Поиск, фильтрация, сортировка товаров (для менеджера и admina)

b. Нефункциональные:
i. Ответ системы — менее 2 секунд
ii. Поддержка до 100 одновременных пользователей
iii. Безопасность: хранение паролей в хэшированном виде
iv. Совместимость с браузерами: Chrome, Firefox, Edge

4. Требования к программной документации

a. ER-диаграмма БД
b. Блок-схема алгоритма
c. Руководство пользователя (в перспективе)

5. Стадии и этапы разработки

a. Модуль 1: Проектирование и создание БД
b. Модуль 2: Разработка Web API
c. Модуль 3: Реализация логики и интерфейса
d. Тестирование, отладка, оформление отчёта

6. Технико-экономические показатели

a. Затраты времени: 80 часов (учебный проект)
b. Используемые технологии: PostgreSQL, ASP.NET Core, C#

7. Стадии контроля и приёмки

a. Промежуточная защита модулей
b. Итоговая защита с демонстрацией работы системы

Структура технического задания

Структура ТЗ соответствует *ГОСТ 19.201-78:*

1. Введение
2. Назначение разработки
   2.1. Цель создания
   2.2. Область применения
   2.3. Характеристика организации-разработчика
3. Требования к системе
   3.1. Функциональные требования
   3.2. Нефункциональные требования
   3.3. Требования к надёжности
   3.4. Требования к безопасности
4. Требования к программной документации
5. Стадии и этапы разработки
6. Порядок контроля и приёмки
7. Приложения
   7.1. ER-диаграмма базы данных
   7.2. Макеты экранов (описание контроллеров)
   7.3. Примеры тестовых данных

Порядок документирования требований

Процесс документирования требований был организован следующим образом:

1. **Сбор требований**
   a. Анализ предметной области (доставка еды, управление заказами)
   b. Изучение аналогичных систем

c. Формализация требований из учебного задания

2. **Формализация и классификация**

   a. Разделение на **функциональные** (что делает система) и **нефункциональные** (как она это делает)
   b. Каждое требование получило идентификатор (например, REQ-001, SEC-002)

3. **Оформление в ТЗ**

   a. Использование шаблона по ГОСТ 19.201-78
   b. Чёткие, краткие формулировки без жаргона
   c. Включение графических приложений (ER-диаграмма, блок-схема)

4. **Верификация**

   a. Проверка полноты и непротиворечивости
   b. Сравнение с исходным заданием
   c. Самотестирование: реализация каждого требования в коде

5. **Поддержка изменений**

   a. При необходимости — внесение правок с указанием версии и даты
   b. Сохранение истории изменений

3.2.1.3. Моделирование процесса разработки программного обеспечения

Анализ требований (определение функциональности и ограничений системы)

На этапе анализа требований была проведена детальная проработка предметной области с целью определения ключевых функций и ограничений будущей системы. Основной задачей стало обеспечение эффективного учёта товаров, заказов и пользователей при соблюдении принципов безопасности, целостности данных и удобства взаимодействия. Были выделены следующие группы требований.

Функциональные требования включали:

- Реализацию механизма авторизации с поддержкой нескольких ролей: гость, клиент, менеджер и администратор.
- Возможность просмотра ассортимента блюд с отображением наименования, описания, категории, производителя, поставщика, цены, наличия скидки и количества на складе.
- Поддержку расширенных возможностей фильтрации, поиска и сортировки для менеджера и администратора.
- Обеспечение возможности добавления, редактирования и удаления товаров только для администратора с контролем доступа.
- Запрет на удаление товара, если он присутствует в оформленных заказах.
- Учёт остатков на складе с автоматическим списанием при оформлении заказа.
- Визуальную индикацию состояния товара: подсветку строк с размером скидки более 15% зелёным цветом, отсутствующих позиций — голубым, а также отображение перечёркнутой старой цены при наличии скидки.

К нефункциональным требованиям отнесены:

- Высокая производительность системы при работе с данными.

- Обеспечение отказоустойчивости за счёт использования нормализованной структуры базы данных третьей нормальной формы.
- Хранение паролей в хэшированном виде для защиты персональных данных.
- Поддержка масштабирования и дальнейшей интеграции с внешними интерфейсами.
- Совместимость с основными веб-браузерами и корректное отображение на различных устройствах.

Особое внимание было уделено ограничениям системы: невозможность одновременного открытия нескольких окон редактирования, обязательная проверка остатков перед добавлением в заказ, а также сохранение параметров сортировки и фильтрации при динамическом поиске. Все требования были задокументированы в соответствии с ГОСТ 19.201-78 и стали основой для последующего проектирования.

Проектирование (разработка архитектуры, дизайна интерфейса и алгоритмов)

Этап проектирования включал разработку логической и физической архитектуры системы, построение ER-диаграммы, создание алгоритмов работы ключевых модулей и формирование структуры пользовательского интерфейса.

Архитектура системы была построена на основе технологии ASP.NET Core Web API с использованием слоистой модели: уровень представления (будущий фронтенд), уровень бизнес-логики, уровень доступа к данным и реляционная база данных PostgreSQL. Вся система была организована в отдельной схеме mealdelivery, что обеспечило логическую изоляцию и упрощённое управление правами доступа.

Логическая модель данных была спроектирована с соблюдением третьей нормальной формы. Основными сущностями стали: пользователи, роли,

категории блюд, производители, поставщики, продукты (готовые обеды), заказы и состав заказов. Связи между таблицами реализованы с помощью внешних ключей с режимами ON DELETE RESTRICT и ON UPDATE CASCADE, что исключает случайное удаление или изменение связанных записей. Для контроля времени изменения данных во всех таблицах предусмотрены поля created_at и updated_at, обновляемые с помощью триггеров.

Для повышения производительности созданы индексы по часто используемым полям: логин пользователя, название товара, категория, количество на складе, статус заказа. Также реализованы пользовательские функции: can_delete_product для проверки возможности удаления товара и calculate_final_price для расчёта итоговой стоимости с учётом скидки.

Проектирование интерфейса выполнено с учётом требований к юзабилити: единый стиль оформления, наличие иконки приложения и логотипа компании, корректное отображение заголовков окон и информации о текущем пользователе. Алгоритмы работы системы оформлены в виде блок-схем согласно ГОСТ 19.701-90, включая процессы авторизации, просмотра товаров, оформления заказа и управления каталогом.

Таким образом, этап проектирования обеспечил чёткую структуру системы, подготовил основу для её реализации и гарантировал соответствие всем поставленным требованиям.

3.2.2. Выполнение интеграции модулей в программное обеспечение

В рамках реализации проекта «Сервис доставки готовых обедов» была разработана и внедрена **модульная архитектура** на основе ASP.NET Core Web API. Это позволило логически разбить приложение на независимые, но взаимосвязанные компоненты, каждый из которых отвечает за определённый

функционал: аутентификация, работа с блюдами, заказами, пользователями и т.д. Важным аспектом стало **четкое разделение ответственности** (Separation of Concerns) и **внедрение зависимостей** (Dependency Injection), что облегчило процесс интеграции новых модулей в уже существующее ПО.

Описание существующей архитектуры

Существующее программное обеспечение состоит из следующих основных модулей:

- **Контроллеры (Controllers)**: AuthController, MealsController, OrdersController, UsersController — обеспечивают обработку HTTP-запросов и взаимодействие с бизнес-логикой.
- **Сервисы (Services)**: TokenService, AuthService — отвечают за генерацию JWT-токенов, аутентификацию и создание гостевых сессий.
- **Репозитории (Repositories)**: IMealRepository, IOrderRepository, IUserRepository и др. — обеспечивают доступ к данным, абстрагируя уровень доступа от бизнес-логики.
- **Сущности (Entities)**: Meal, Order, User, Category и т.д. — представляют собой модели данных, соответствующие таблицам в БД.
- **DTO (Data Transfer Objects)**: MealResponse, OrderResponse, LoginRequest и т.п. — используются для передачи данных между слоями и во внешний интерфейс.
- **Контекст данных (MealDeliveryDbContext)**: обеспечивает связь с PostgreSQL через Entity Framework Core.
- **Middleware**: ExceptionHandlingMiddleware — обрабатывает глобальные ошибки приложения.

- **Конфигурация и расширения**: Методы в ServiceExtensions и Program.cs регистрируют все сервисы, настраивают аутентификацию, CORS, Swagger и т.д.

Все модули связаны через интерфейсы и внедрение зависимостей, что делает систему **гибкой и тестируемой**.

Создание и интеграция нового модуля: модуль статистики заказов

Для расширения функциональности системы был разработан и интегрирован **новый модуль статистики заказов**, предназначенный для менеджеров и администраторов. Он позволяет получать аналитические данные: общее количество заказов, выручка, средний чек, количество заказов за день, распределение по статусам.

Процесс интеграции:

1. **Определение функционала модуля**:
    a. Получение общей статистики по заказам.
    b. Возврат данных в формате JSON через API-метод.
2. **Разработка DTO**:
    a. Созданы классы OrderStatistics и OrderStatisticsResponse в MealDelivery.Models.DTOs.Responses для представления данных статистики.
3. **Расширение интерфейса репозитория**:
    a. В интерфейс IOrderRepository добавлен метод Task<OrderStatistics> GetOrderStatisticsAsync().

4. **Реализация логики в репозитории**:
    a. В классе OrderRepository реализован метод GetOrderStatisticsAsync, который:
        i. Выполняет подсчёты с помощью LINQ и EF.Functions (например, COUNT, SUM, GROUP BY).
        ii. Использует DateTime.UtcNow для фильтрации заказов за текущий день.
        iii. Возвращает объект OrderStatistics с заполненными полями.
5. **Интеграция в контроллер**:
    a. В OrdersController добавлен новый метод GET /api/orders/statistics:
        i. Аннотирован атрибутом [Authorize(Roles = "manager, admin")], ограничивая доступ.
        ii. Вызывает IOrderRepository.GetOrderStatisticsAsync().
        iii. Возвращает результат в формате ApiResponse<OrderStatisticsResponse>.
6. **Регистрация зависимостей**:
    a. Новый функционал автоматически интегрирован через существующую систему внедрения зависимостей: IOrderRepository уже зарегистрирован, и новый метод доступен через DI.
7. **Тестирование и документирование**:
    a. Метод доступен через Swagger UI.
    b. Обрабатывает ошибки через ExceptionHandlingMiddleware.

Результат интеграции:

Новый модуль стал полноценной частью системы. Он **не нарушил структуру** существующего кода, **соответствует архитектуре**, использует общие шаблоны обработки ошибок и форматирования ответов, и **безопасно интегрирован** через систему авторизации. Это подтверждает, что выбранная архитектура позволяет **масштабировать приложение** за счёт добавления новых модулей с минимальными изменениями в остальные части системы.

3.2.3. Выполнение отладки программного модуля с использованием специализированных программных средств

В рамках реализации проекта «Сервис доставки готовых обедов» процесс отладки программного модуля проводился с использованием специализированных средств, встроенных в среду разработки Visual Studio. Основной задачей отладки являлось выявление, локализация и устранение потенциальных ошибок в коде, а также проверка корректности работы бизнес-логики и взаимодействия компонентов системы.

Отладка велась преимущественно на уровне контроллеров Web API, репозиториев и сервисов. Для начала работы был запущен проект в режиме отладки (Debug), что позволило системе автоматически приостановить выполнение кода в заданных точках останова (breakpoints). Такие точки устанавливались в критических участках кода, например, при обработке запроса на создание заказа, в методах, изменяющих остатки блюд, или при проверке возможности удаления товара, связанного с заказами.

Во время выполнения кода использовались встроенные инструменты Visual Studio: окно «Autos» и «Locals» для просмотра значений переменных, окно «Watch» для отслеживания состояния сложных объектов, таких как Order или Meal, а также вызовов асинхронных методов. Это позволяло в

реальном времени анализировать, как данные проходят через слои приложения — от контроллера к сервису, а затем к репозиторию и далее в базу данных.

Особое внимание уделялось проверке корректности работы механизма авторизации и разграничения ролей. Здесь также использовались точки останова в методах контроллеров, защищённых атрибутами [Authorize] и [Authorize(Roles = "...")], чтобы убедиться, что доступ к определённым операциям действительно ограничен в зависимости от токена, передаваемого в заголовке запроса.

Для проверки взаимодействия с базой данных применялась отладка на уровне репозиториев. Благодаря использованию Entity Framework Core, в процессе отладки можно было наблюдать, как формируются и выполняются SQL-запросы, проверять корректность JOIN-ов, фильтрации и сортировки, а также убеждаться в правильности обновления и сохранения данных. Например, при тестировании метода CanDeleteMealAsync, проверялось, что запрос к таблице OrderDetails действительно возвращает false, если блюдо присутствует в каком-либо заказе.

Также использовался механизм логирования, встроенный в ASP.NET Core. В процессе отладки в консоль выводились сообщения о начале и завершении выполнения методов, а также о возникновении потенциальных ошибок, что дополнительно помогало отследить логику выполнения и упростить диагностику.

Таким образом, отладка с использованием Visual Studio позволила не только локализовать и устранить логические и синтаксические ошибки, но и подтвердить корректность реализации ключевых функций приложения, включая работу с сущностями, проверку бизнес-правил и безопасное взаимодействие с данными.

3.2.4. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

В рамках реализации проекта «Сервис доставки готовых обедов» был проведён комплекс мероприятий по обеспечению качества программного обеспечения. Основной задачей стало выявление потенциальных ошибок на ранних этапах, проверка корректности бизнес-логики и удостоверение в соответствии требованиям, предъявленным в техническом задании. Процесс тестирования включал в себя разработку тестовых сценариев и их последующую проверку с использованием ручных и автоматизированных методов.

Разработка тестовых сценариев

Тестовые сценарии были сформированы на основе анализа функциональных требований, таких как:

- Работа с ролями пользователей (гость, клиент, менеджер, администратор),
- Управление блюдами (просмотр, фильтрация, поиск, добавление, редактирование, удаление),
- Управление заказами (оформление, просмотр, изменение статуса),
- Учёт остатков на складе,
- Защита от удаления блюда, если оно присутствует в заказе.

Для каждого функционального модуля были выделены ключевые сценарии, описывающие как **ожидаемое поведение** системы при корректных данных, так и **реакцию** на некорректные или граничные условия. Примеры сценариев:

1. **Сценарий: Авторизация администратора**
    a. **Предусловие:** В БД существует пользователь с ролью admin.

b. **Шаги:**
    i. Отправить POST-запрос на /api/auth/login с логином и паролем администратора.
    ii. Проверить наличие токена в ответе.
    iii. Использовать токен для вызова защищённого эндпоинта (например, /api/meals с методом POST).
c. **Ожидаемый результат:** Успешная аутентификация, возврат токена, разрешён доступ к защищённому ресурсу.

2. **Сценарий: Попытка удаления блюда, присутствующего в заказе**
    a. **Предусловие:** В системе есть блюдо, которое включено в один или несколько заказов.
    b. **Шаги:**
        i. Авторизоваться как администратор.
        ii. Отправить DELETE-запрос на /api/meals/{id}, где id — ID блюда из заказа.
    c. **Ожидаемый результат:** Ответ 400 Bad Request с сообщением "Нельзя удалить блюдо, которое есть в заказах".

3. **Сценарий: Фильтрация блюд менеджером**
    a. **Предусловие:** В системе есть несколько блюд с разными производителями.
    b. **Шаги:**
        i. Авторизоваться как менеджер.
        ii. Отправить GET-запрос на /api/meals?manufacturerId={id}.
    c. **Ожидаемый результат:** В ответе — только блюда выбранного производителя.

4. **Сценарий: Оформление заказа с недостаточным остатком**

    a. **Предусловие:** В наличии есть блюдо с stock_quantity = 2.
    b. **Шаги:**
        i. Авторизоваться как клиент.
        ii. Отправить POST-запрос на /api/orders с 3-мя единицами этого блюда.
    c. **Ожидаемый результат:** Ответ 400 Bad Request с сообщением о недостатке товара.

Процесс тестирования

Тестирование проводилось в несколько этапов:

1. **Модульное тестирование (Unit Testing)** (при наличии): Проверялись отдельные методы в репозиториях, сервисах и контроллерах на корректность возвращаемых значений, обработку исключений и взаимодействие с зависимостями. Это позволило изолированно протестировать бизнес-логику, например, метод CanDeleteMealAsync.

2. **Интеграционное тестирование (Integration Testing)**: Проверялось взаимодействие между различными уровнями приложения — контроллером, сервисом и репозиторием — с использованием реальной базы данных или моков. Например, проверка, что при создании заказа корректно списывается количество блюд из stock_quantity.

3. **Функциональное тестирование (Manual/Postman)**: С помощью инструмента **Postman** были созданы коллекции запросов, соответствующие тестовым сценариям. Это позволило проверить:

    a. Работу авторизации и разграничения ролей.
    b. Фильтрацию, поиск и сортировку блюд.

c. Оформление и просмотр заказов.
d. Обновление статусов заказов.
e. Валидацию входных данных (например, отрицательная цена, превышение остатка).

4. **Тестирование безопасности**: Проверялось, что:

   a. Неавторизованный пользователь не может получить доступ к защищённым ресурсам.
   b. Пользователь с ролью client не может вызвать эндпоинты администратора.
   c. Неверные токены или истёкшие токены возвращают 401 Unauthorized.

5. **Тестирование производительности и стабильности**: Проверялась реакция приложения при одновременном выполнении нескольких запросов, корректная работа в течение длительного времени, отсутствие утечек памяти.

Результаты тестирования

В ходе тестирования были выявлены и устранены:

- Ошибки валидации данных (например, отрицательные цены).
- Проблемы с неправильной сортировкой блюд.
- Нарушения целостности данных при удалении блюд.
- Некорректная работа с изображениями (ошибки при загрузке и удалении файлов).

Также была проверена и подтверждена корректная работа механизма обновления полей updated_at с помощью триггеров, а также расчёт финальной цены с учётом скидки.

Таким образом, разработка и выполнение тестовых сценариев позволили обеспечить высокое качество программного обеспечения, подтвердить его соответствие требованиям и повысить уверенность в стабильной работе системы в реальных условиях эксплуатации.

3.2.5. Осуществление инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

Изучение регламентирующих документов и анализ соответствия кода стандартам

В рамках реализации проекта «Сервис доставки готовых обедов» особое внимание уделялось не только функциональной стороне разработки, но и соблюдению современных стандартов кодирования, правил оформления и структурирования программного обеспечения. Это позволило повысить читаемость, поддерживаемость и надёжность кода, а также облегчило его сопровождение и дальнейшее развитие.

1. Регламентирующие документы и стандарты

Хотя в ООО «Эльбрус» не использовалась строго формализованная корпоративная политика кодирования (так как основной деятельностью является производство перегородок), при разработке программного обеспечения были применены общеизвестные и общепринятые стандарты и практики, рекомендуемые сообществом .NET и PostgreSQL:

- Стиль кодирования C# (C# Coding Conventions):

    - Использование PascalCase для имён классов, свойств, методов и публичных членов (например, MealDeliveryDbContext, GetFilteredMealsAsync).
    - Использование camelCase для имён переменных, параметров и приватных полей (например, _context, searchTerm).
    - Имена интерфейсов начинаются с буквы I (например, IMealRepository).
    - Структура файлов соответствует принципу один класс — один файл, что улучшает читаемость и навигацию.
- Стандарты проектирования (SOLID, DRY, KISS, YAGNI):
    - Применение принципа разделения ответственности: код разделён на слои (контроллеры, сервисы, репозитории, сущности), каждый из которых отвечает за свою задачу.
    - Использование внедрения зависимостей (DI) через Program.cs и ServiceExtensions, что делает систему гибкой и тестируемой.
    - Принцип DRY (Don't Repeat Yourself) соблюдается за счёт использования общих DTO, базовых классов репозиториев (GenericRepository) и общих функций в Meal.
- Структура проекта (ASP.NET Core Web API):
    - Проект организован в виде решения (.sln) с отдельными проектами: MealDelivery.Api (основной Web API) и MealDelivery.Core (сущности, DTO, перечисления).
    - Код разбит на логические папки: Controllers, Services, Repositories, Models, Data, Middleware, что соответствует общепринятой структуре ASP.NET Core приложений.
- Стандарты SQL и проектирования БД (PostgreSQL):
    - Использование snake_case для имён таблиц и столбцов (например, meals, stock_quantity) — это принятый стиль в PostgreSQL.

    - Соблюдение третьей нормальной формы (3НФ), отсутствие избыточности данных.
    - Использование ENUM-типов (user_role, order_status) для строгой типизации и целостности данных.
    - Применение ограничений (CHECK, NOT NULL, UNIQUE) и внешних ключей (FK) с корректной стратегией ON DELETE (RESTRICT, CASCADE, SET NULL).
- Безопасность:
    - Пароли хранятся в хэшированном виде с помощью BCrypt.Net.
    - Используется JWT-аутентификация с настроенной валидацией токенов.
    - Применение атрибутов [Authorize] и [Authorize(Roles = ...)] для контроля доступа.
- Документирование:
    - Использование XML-комментариев к контроллерам и методам, которые интегрируются в Swagger.
    - Комментарии к схеме и таблицам в SQL-скриптах.

2. Анализ соответствия кода принятым стандартам

Анализ кода, представленного в файле Код пп0201 Бушмакина.txt, показывает, что:

- Структура проекта строго соответствует архитектуре ASP.NET Core Web API с использованием слоёв и внедрения зависимостей.
- Контроллеры (AuthController, MealsController, OrdersController) содержат методы, соответствующие REST-подходу и защите по ролям.
- Репозитории (IMealRepository, OrderRepository и др.) реализуют паттерн Repository, абстрагируя уровень доступа к данным. Используется GenericRepository для базовых операций.

- Сущности (Meal, Order, User) соответствуют структуре таблиц в БД, используют атрибуты Data Annotations и NotMapped для логики, не требующей сохранения.
- DTO (Data Transfer Objects) используются для передачи данных между слоями, что обеспечивает чистоту модели и защиту от overposting.
- Middleware (ExceptionHandlingMiddleware) централизованно обрабатывает исключения, формируя структурированные ответы об ошибках.
- SQL-скрипты (из БД пп0201.docx) были адаптированы под тему проекта, но сохраняют ту же структуру, стиль именования и логику, что и исходный шаблон книжного магазина. Это подтверждает, что принципы проектирования БД и стандарты SQL соблюдены.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 03 по 13 ноября 2025 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. Разработана и реализована полнофункциональная база данных для сервиса доставки готовых обедов в СУБД PostgreSQL, полностью соответствующая требованиям третьей нормальной формы, с обеспечением ссылочной целостности, использованием перечислений (ENUM) для ролей и статусов, а также с применением триггеров и функций для автоматизации контроля данных — включая запрет удаления товара при наличии в заказах и обновление меток времени изменений.

2. Создана архитектура программного решения на основе ASP.NET Core Web API, включающая структурированную модель данных, реализацию ролевого доступа, корректную обработку запросов на фильтрацию, поиск и сортировку товаров в реальном времени, а также безопасное управление изображениями блюд с автоматическим сохранением и удалением файлов.

3. Разработано и документировано техническое задание на информационную систему в соответствии с ГОСТ 19.201-78 и ГОСТ 34.601-90, что позволило систематизировать требования, чётко определить функциональные и нефункциональные характеристики системы, а также зафиксировать этапы разработки и критерии приёмки.

4. Проведён детальный анализ и моделирование бизнес-процессов в состоянии «AS IS» и «TO BE», что продемонстрировало значительное повышение эффективности управления ассортиментом и заказами за счёт автоматизации, исключения ручных операций и снижения рисков ошибок.

5. Сформирована полная техническая документация, включающая ER-диаграмму, SQL-скрипты создания БД, описание логических моделей, блок-схемы алгоритмов и пояснительную записку, готовую к использованию как в учебных, так и в реальных производственных условиях.

6. Получены практические навыки проектирования, разработки и внедрения информационных систем в условиях реального предприятия — ООО «Эльбрус» — что позволило успешно применить теоретические знания к решению задач, выходящих за рамки учебного задания и соответствующих современным стандартам цифровой трансформации бизнеса.

По окончании практики был получен практический опыт в области технологий разработки программного обеспечения на основе анализа проектной и технической документации, а также реализации прототипа информационной системы. Были изучены и применены современные методы проектирования баз данных, разработки архитектуры приложения с использованием ASP.NET Core Web API, а также стандарты оформления документации в соответствии с ГОСТ 19 и ГОСТ 34. В ходе работы были задействованы такие инструменты, как СУБД PostgreSQL, Visual Studio, языки SQL и C#.

Выполнены следующие задачи:

- Проанализированы требования к информационной системе, определены функциональные и нефункциональные характеристики будущего ПО.

- Спроектирована логическая и физическая структура базы данных, разработана ER-диаграмма, обеспечивающая нормализацию данных до 3НФ и целостность связей.

- Создан SQL-скрипт для инициализации структуры БД в PostgreSQL, включающий таблицы, внешние ключи, ограничения, индексы, ENUM-типы, триггеры и пользовательские функции.

- Разработана архитектура приложения на основе ASP.NET Core Web API, включая отдельную библиотеку классов для моделей.

- Реализованы контроллеры и сервисы для обеспечения функциональности, соответствующей требованиям различных ролей пользователей (гость, клиент, менеджер, администратор).

- Обеспечена реализация бизнес-логики, включая учёт остатков, запрет на удаление товара при наличии в заказах, визуальную индикацию скидок и отсутствия товара.

- Подготовлено техническое задание, соответствующее требованиям ГОСТ 19.201-78, включающее полное описание системы, требования, стадии и этапы разработки.

- Проведено моделирование бизнес-процессов в состоянии «AS IS» и «TO BE» с последующим сравнительным анализом.

- Созданы блок-схемы алгоритмов работы ключевых модулей системы в соответствии с ГОСТ 19.701-90.

- Оформлена полная документация по проекту, включая пояснительную записку, скриншоты, SQL-скрипты и описания.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1 Стружкин, Н. П. Базы данных: проектирование. Практикум: учебное пособие для среднего профессионального образования / Н. П. Стружкин, В. В. Годин. Москва : Издательство Юрайт, 2019. — 291 с. — (Профессиональное образование). — ISBN 978-5-534-08140-4. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/442343

2 Илюшечкин, В. М. Основы использования и проектирования баз данных : учебник

для среднего профессионального образования / В. М. Илюшечкин. испр. и доп. —

Москва : Издательство Юрайт, 2019. — 213 с. — (Профессиональное образование). —

ISBN 978-5-534-01283-5. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/437670

Дополнительные источники

3 Советов, Б. Я. Базы данных : учебник для среднего профессионального образования / Б. Я. Советов, В. В. Цехановский, В. Д. Чертовской. 3-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2019. — 420 с. — (Профессиональное образование). — ISBN 978-5-534-09324-7. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/438438

4 Нестеров, С. А. Базы данных : учебник и практикум для среднего профессионального образования / С. А. Нестеров. Москва : Издательство Юрайт, 2020. — 230 с. — (Профессиональное образование). — ISBN 978-5-534-11629-8. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/457142

5 Стасышин, В. М. Базы данных: технологии доступа : учебное пособие для среднего профессионального образования / В. М. Стасышин, Т. Л. Стасышина 2-е изд., испр. и доп. — Москва : Издательство Юрайт, 2019. — 164 с. — (Профессиональное образование). — ISBN 978-5-534-09888-4. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/442342

6 Голицына, О. Л. Основы проектирования баз данных : учеб. пособие / О.Л. Голицына, Т.Л. Партыка, И.И. Попов. 2-е изд., перераб. и доп. — Москва : ФОРУМ : ИНФРА-М, 2019. — 416 с. : ил. — (Среднее профессиональное образование). - ISBN 978-5-16- 105762-9. - Текст : электронный. - URL: https://znanium.com/catalog/product/1018906

7 Стружкин, Н. П. Базы данных: проектирование : учебник для среднего профессионального образования / Н. П. Стружкин, В. В. Годин. Москва : Издательство Юрайт, 2020. — 477 с. — (Профессиональное образование). — ISBN 978-5-534-11635-9. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/457135

8 Партыка, Т. Л. Информационная безопасность : учеб. пособие / Т.Л. Партыка, И.И. Попов5-е изд., перераб. и доп. — Москва : ФОРУМ : ИНФРА-М, 2018. — 432 с. — (Среднее профессиональное образование). - ISBN 978-5-16-101302-1. - Текст: электронный. - URL: https://znanium.com/catalog/product/915902

9 Грекул , В. И. Проектирование информационных систем : учебник и практикум для среднего профессионального образования / В. И. Грекул, Н. Л. Коровкина, Г. А. Левочкина. Москва : Издательство Юрайт, 2020. — 385 с. — (Профессиональное образование). — ISBN 978-5-534-12104-9. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/457223